

Stack vs Heap

Memory Management in C++

In-Depth Study Guide · All Tasks Solved · Interview Prep

PHASE	Week 1 — C++ Core
TOPIC	Memory Management
PREREQ	Day 1 — Pointers & References
TOOLS	g++, valgrind, ASan
KEY LIBS	, , sanitizer
DIFFICULTY	Intermediate

■ Day 2 Objective

Understand where your data lives in memory. The stack is fast but limited; the heap is flexible but manually managed. Mismanage it and your search engine crawler will silently corrupt memory at scale.

By end of day: you will detect, reproduce, fix, and prevent every common memory bug in C++ — leaks, overflows, dangling pointers, and stack overflows.

Section	Topic
1	Core Concepts — Stack vs Heap (Deep Dive)
2	Stack Frame Lifecycle & Memory Layout
3	Heap Allocation: new / delete
4	Memory Leaks — Detection & Prevention
5	Buffer Overflow — Silent Corruption
6	Fragmentation & Allocators

7	AddressSanitizer (ASan)
8	TASK 1 — Intentional Memory Leak + Valgrind
9	TASK 2 — Fix with delete[] + Valgrind
10	TASK 3 — Stack-Based String Class + Stack Overflow
11	TASK 4 — ASan Buffer Overflow Detection
12	Memory Comparison Quick-Reference
13	Interview Questions (30 Q&A)

1. Core Concepts — Stack vs Heap

MEMORY ARCHITECTURE

Every C++ program divides its memory into several regions. The two you interact with most are the **stack** and the **heap**. Understanding them is not optional for systems programming — they determine performance, safety, and the correctness of everything you build.

Property	Stack	Heap
Allocation Speed	~1 ns (just move SP)	~100 ns (malloc bookkeeping)
Size	8 MB default (Linux)	Limited only by RAM/swap
Management	Automatic (compiler)	Manual (programmer)
Lifetime	Until scope exits	Until explicitly freed
Thread Safety	Private per thread	Shared — needs locking
Fragmentation	None	Internal + External
Cache Locality	Excellent	Poor (scattered)
Typical Use	Local vars, params, frames	Dynamic data, large arrays
Overflow Risk	Stack overflow (recursion)	Heap corruption
Dealloc. Order	LIFO — guaranteed	Any order

2. Stack Frame Lifecycle & Memory Layout

STACK INTERNALS

When a function is called, the CPU pushes a **stack frame** onto the stack. Each frame contains: the return address, saved registers, local variables, and the previous frame pointer. On x86-64 Linux, the stack grows **downward** (from high addresses toward low addresses).

What a Stack Frame Contains:

- **Return Address** — where execution resumes after the function returns
- **Saved rbp** — caller's frame pointer, so it can be restored
- **Local Variables** — allocated contiguously at fixed offsets from rbp
- **Function Arguments** — first 6 go in registers (rdi, rsi, rdx, rcx, r8, r9); rest on stack
- **Padding / Alignment** — stack must be 16-byte aligned before a call (ABI requirement)

Stack Frame — Code Example:

stack_frame_demo.cpp

```
// Compile: g++ -O0 -g stack_demo.cpp -o stack_demo
// Disassemble: objdump -d stack_demo | grep -A30 ''

#include
```

```
using namespace std;

int foo(int x, int y) {
    int local_a = x + y; // lives at [rbp - 4]
    int local_b = x * y; // lives at [rbp - 8]
    char buf[64]; // lives at [rbp - 72]
    return local_a + local_b;
} // stack frame destroyed HERE — all locals gone

int main() {
    int result = foo(3, 4); // pushes frame, passes 3→rdi, 4→rsi
    cout << result << endl; // 19
    return 0;
}
```

Assembly insight (x86-64 — what the compiler generates):

foo assembly (simplified)

```
; Entry to foo:
push rbp ; save caller's frame pointer
mov rbp, rsp ; our frame pointer = current stack top
sub rsp, 80 ; reserve space: 4+4+64 bytes (aligned to 16)

; local_a = x + y
mov eax, edi ; edi = x (first arg)
add eax, esi ; esi = y (second arg)
mov DWORD PTR [rbp-4], eax

; return
pop rbp ; restore caller's frame pointer
ret ; jump to return address
```

■ Key Insight — Why Stack is Fast

Stack allocation is literally ONE instruction: SUB RSP, N. There is no bookkeeping, no free-list traversal, no lock. This is why stack-allocated local variables are ~100x faster to allocate than heap.

The price: limited size (default 8 MB on Linux) and no persistence beyond the scope.

3. Heap Allocation: new / delete

HEAP INTERNALS

The heap (also called the free store in C++) is the region used for dynamic memory. `new` calls operator `new` which ultimately calls `malloc()`, which uses system calls like `brk()` or `mmap()` to get memory from the OS. The allocator maintains a **free-list** data structure to track available blocks.

The new / delete Ecosystem:

heap_basics.cpp


```
delete[] buf; // likely crashes inside free()
}
```

■ Why This Matters for Your Crawler

Your C++ crawler parses untrusted HTML. If you use fixed-size buffers and copy user-controlled data (URLs, header values) without bounds checking, an attacker can craft an HTTP response that crashes or compromises your crawler.

Always use `std::string`, `std::vector`, or `strncpy` with explicit size limits.

6. Fragmentation & Allocators

FRAGMENTATION

Fragmentation is the waste of memory due to poor allocation patterns. It does not cause crashes directly, but degrades performance and can cause allocations to fail even when total free memory exceeds the request size.

Type	Definition	Example
Internal	Wasted space INSIDE an allocated block (alignment padding)	Request 9 bytes, get 16 (aligned to 8)
External	Free blocks too small to satisfy request despite total space being sufficient	50 free 4-byte gaps, can't serve 8-byte request

Slab / Pool Allocators (Production Solution):

pool_allocator.cpp

```
// Pool allocator: pre-allocates a pool, hands out fixed-size chunks
// Zero fragmentation, O(1) alloc/free, cache-friendly

template
class PoolAllocator {
    alignas(T) char pool[sizeof(T) * PoolSize];
    T* free_list[PoolSize];
    size_t free_count;
public:
    PoolAllocator() : free_count(PoolSize) {
        for (size_t i = 0; i < PoolSize; i++)
            free_list[i] = reinterpret_cast(pool + i * sizeof(T));
    }
    T* allocate() {
        if (free_count == 0) throw std::bad_alloc{};
        return free_list[--free_count];
    }
    void deallocate(T* p) { free_list[free_count++] = p; }
```

```
};  
// Used in your URL queue: pool-allocate URLEntry structs
```

7. AddressSanitizer (ASan) — Runtime Detection

ADDRESSSANITIZER

AddressSanitizer is a compile-time instrumentation tool by Google. It inserts shadow memory checks around every heap/stack access. It catches: heap-buffer-overflow, stack-buffer-overflow, use-after-free, double-free, use-after-return, and memory leaks — all with ~2x runtime overhead.

Enabling ASan:

ASan compile commands

```
# Compile with ASan:  
g++ -fsanitize=address -fno-omit-frame-pointer -g -O1 program.cpp -o program  
  
# Run — ASan output automatically printed on error:  
./program  
  
# Also enable leak detection explicitly:  
ASAN_OPTIONS=detect_leaks=1 ./program  
  
# Combine with UBSan (undefined behavior sanitizer):  
g++ -fsanitize=address,undefined -g program.cpp -o program
```

Sample ASan Output for a Heap-Buffer-Overflow:

ASan output

```
=====
==12345==ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 1 at 0x602000000050 thread T0
#0 0x401234 in main overflow.cpp:8
#1 0x7f... in __libc_start_main

0x602000000050 is located 0 bytes to the right of 8-byte region
[0x602000000048,0x602000000050)
allocated by thread T0 here:
#0 0x... in operator new[](unsigned long)
#1 0x401200 in main overflow.cpp:6

SUMMARY: AddressSanitizer: heap-buffer-overflow overflow.cpp:8
```

■ ASan vs Valgrind

ASan: 2x overhead, compile-time instrumentation, catches stack bugs too, immediate abort on error, production-like speed.

Valgrind: 10-50x overhead, no recompilation needed, works on any binary, better for gradual memory analysis, not suitable for performance benchmarks.

Use BOTH: ASan during development, Valgrind for final verification.


```
}

int main() {
    cout << "=== Running leaky program ===" << endl;
    simulate_leak(); // leaks 4000 bytes
    nested_leak(); // leaks 256 + 3*Node bytes
    cout << "Program ended. Check valgrind output." << endl;
    return 0;
}
```

Valgrind Output (Leak Detected):

valgrind output — leaks detected

```
$ valgrind --leak-check=full --show-leak-kinds=all ./memory_leak

==PID== HEAP SUMMARY:
==PID== in use at exit: 4,280 bytes in 5 blocks
==PID== total heap usage: 7 allocs, 2 frees, 78,992 bytes allocated

==PID== 4,000 bytes in 1 blocks are definitely lost in loss record 1 of 5
==PID== at 0x...: operator new[](unsigned long) (vg_replace_malloc.c:640)
==PID== by 0x...: simulate_leak() (memory_leak.cpp:14)
==PID== by 0x...: main (memory_leak.cpp:40)

==PID== 256 bytes in 1 blocks are definitely lost in loss record 4 of 5
==PID== at 0x...: operator new[](unsigned long) (vg_replace_malloc.c:640)
==PID== by 0x...: nested_leak() (memory_leak.cpp:25)

==PID== LEAK SUMMARY:
==PID== definitely lost: 4,280 bytes in 5 blocks
==PID== indirectly lost: 0 bytes in 0 blocks
==PID== possibly lost: 0 bytes in 0 blocks
==PID== still reachable: 0 bytes in 0 blocks
==PID== ERROR SUMMARY: 5 errors from 5 contexts
```

9. TASK 2 — Fix with delete[] + Valgrind Verification

memory_fixed.cpp — FIXED VERSION

```
// FILE: memory_fixed.cpp

// Compile: g++ -g -O0 memory_fixed.cpp -o memory_fixed

// Verify: valgrind --leak-check=full ./memory_fixed

#include
#include

using namespace std;

struct Node { int val; Node* next; };
```

```
// Helper: free entire linked list
void free_list(Node* head) {
    while (head) {
        Node* tmp = head->next;
        delete head; // free current node
        head = tmp; // move to next
    }
}

void simulate_no_leak() {
    int* arr = new int[1000];
    for (int i = 0; i < 1000; i++) arr[i] = i * 2;
    cout << "arr[500] = " << arr[500] << endl;
    delete[] arr; // ← FIXED: array delete
}

void nested_no_leak() {
    char* str = new char[256];
    strcpy(str, "This string is NOT leaked");
    cout << str << endl;
    delete[] str; // ← FIXED

    Node* head = new Node{1, nullptr};
    head->next = new Node{2, nullptr};
    head->next->next = new Node{3, nullptr};
    free_list(head); // ← FIXED: recursive free
}

int main() {
    simulate_no_leak();
    nested_no_leak();
    cout << "All memory freed cleanly." << endl;
    return 0;
}
```

Valgrind Output After Fix:

valgrind output — CLEAN

```
$ valgrind --leak-check=full ./memory_fixed

==PID== HEAP SUMMARY:
==PID== in use at exit: 0 bytes in 0 blocks
==PID== total heap usage: 7 allocs, 7 frees, 78,992 bytes allocated

==PID== All heap blocks were freed -- no leaks are possible
```

```
==PID== ERROR SUMMARY: 0 errors from 0 contexts
```

■ Key Insight

The total allocs (7) now equals total frees (7). Valgrind reports '0 bytes in use at exit'. Every `new[]` is paired with exactly one `delete[]`.

10. TASK 3 — Stack-Based String Class + Stack Overflow Demo

TASK 3: STACK STRING + OVERFLOW

Part A — Stack-Based String Class:

stack_string.cpp

```
// FILE: stack_string.cpp

// Compile: g++ -g -O0 stack_string.cpp -o stack_string

#include
#include
#include

using namespace std;

// A fixed-capacity string that lives entirely on the stack.
// No heap allocation at all – extremely cache-friendly.

template
class StackString {

    char data[MaxLen + 1]; // +1 for null terminator
    size_t len;

public:
    StackString() : len(0) { data[0] = '\0'; }

    // Constructor from C-string
    explicit StackString(const char* s) {
        len = strlen(s);
        if (len > MaxLen)
            throw overflow_error("StackString overflow");
        memcpy(data, s, len + 1);
    }

    // Append
    StackString& append(const char* s) {
        size_t slen = strlen(s);
        if (len + slen > MaxLen)
            throw overflow_error("StackString overflow on append");
        memcpy(data + len, s, slen + 1);
        len += slen;
        return *this;
    }

    const char* c_str() const { return data; }
```

```
size_t size() const { return len; }
bool empty() const { return len == 0; }

// Subscript with bounds checking
char operator[](size_t i) const {
    if (i >= len) throw out_of_range("index out of range");
    return data[i];
}

};

int main() {
    StackString<64> s1("Hello, ");
    s1.append("Search Engine!");
    cout << s1.c_str() << " (len=" << s1.size() << ")" << endl;

    // All memory is on the stack – no malloc, no new
    // When main() returns, s1's storage is instantly reclaimed
    return 0;
}
```

Part B — Stack Overflow via Deep Recursion:

[stack_overflow.cpp](#)

```
// FILE: stack_overflow.cpp
// Compile: g++ -g -O0 stack_overflow.cpp -o stack_overflow
// WARNING: This WILL crash – run in a safe environment

#include
using namespace std;

int depth = 0;

// Each call: ~100 bytes on stack (local vars + frame overhead)
// Stack limit: 8 MB = ~80,000 frames before SIGSEGV
void recursive(int n) {
    char padding[100]; // force each frame to consume 100 bytes
    padding[0] = n; // prevent optimization
    depth++;

    if (depth % 10000 == 0)
        cout << "Depth: " << depth << endl;

    recursive(n + 1); // tail call: but NOT optimized (O0)
}

int main() {
    cout << "Starting deep recursion..." << endl;
```

```
try {
    recursive(0);
} catch (...) {
    // Note: stack overflow is SIGSEGV, not a C++ exception!
    // This catch will NOT fire — the OS sends SIGSEGV instead.
    cout << "Caught exception" << endl;
}
return 0; // Never reached
}

// Expected output: Depth: 10000 → 20000 → ... → Segmentation fault (core dumped)
// Check stack limit: ulimit -s (typically 8192 KB)
// Increase limit: ulimit -s unlimited (for testing only!)
```

■■ Preventing Stack Overflow in Real Code

1. Convert deep recursion to iterative (use `std::stack` for DFS).
2. Use `ulimit -s unlimited` carefully — only for known-deep algorithms.
3. For crawler DFS: use an explicit stack on the heap, not call-stack recursion.
4. In production, limit recursion depth: if `(depth > MAX_DEPTH)` return;


```
int main(int argc, char* argv[]) {
    int demo = (argc > 1) ? atoi(argv[1]) : 1;
    switch (demo) {
        case 1: heap_oob(); break;
        case 2: use_after_free(); break;
        case 3: stack_oob(); break;
        case 4: double_free(); break;
        default: cout << "Usage: ./asan_demo [1-4]" << endl;
    }
    return 0;
}
```

Expected ASan Output for Demo 1 (heap-buffer-overflow):

ASan output — heap-buffer-overflow

```
$ ./asan_demo 1

=====
==PID==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x...
WRITE of size 1 at 0x602000000058 thread T0
#0 0x401234 in heap_oob() asan_demo.cpp:14
#1 0x4013ac in main asan_demo.cpp:42

0x602000000058 is located 0 bytes to the right of 8-byte region
SUMMARY: AddressSanitizer: heap-buffer-overflow asan_demo.cpp:14 heap_oob()
```

12. Memory Quick-Reference & Cheat Sheet

CHEAT SHEET

Operation	C++ Code	Notes
Scalar alloc	<code>int* p = new int(5);</code>	Use delete p
Array alloc	<code>int* a = new int[N];</code>	Use delete[] a
Zero-init array	<code>int* a = new int[N]();</code>	Parentheses = value-init
Free scalar	<code>delete p; p = nullptr;</code>	Set to nullptr after
Free array	<code>delete[] a; a = nullptr;</code>	MUST be delete[]
Valgrind	<code>valgrind --leak-check=full ./prog</code>	Best for production audit
ASan compile	<code>g++ -fsanitize=address -g ...</code>	2x overhead, catches more
Stack limit	<code>ulimit -s</code>	Typically 8192 KB
Check heap size	<code>valgrind --tool=massif ./prog</code>	Heap profiler

13. Interview Questions — Full Q&A;

30 INTERVIEW QUESTIONS

■ Question Categories

Conceptual (1–10): Definitions, comparisons, mental models.

Technical (11–20): Code tracing, debugging, internals.

Scenario / Design (21–26): System design, trade-off decisions.

Advanced (27–30): Production systems, OS internals, compilers.

Conceptual Questions

Q1

■ Easy

What is the difference between stack and heap memory?

Stack: LIFO memory region managed automatically by the compiler/CPU. Used for local variables, function parameters, and return addresses. Allocation is ~1 instruction (SUB RSP, N). Size is typically 8 MB.

Heap: Dynamic memory region managed manually by the programmer. Allocation via new/malloc involves bookkeeping (~100 ns). Size limited only by available RAM. Lives until explicitly freed.

Key difference: stack variables are destroyed when their scope exits; heap variables persist until delete/free is called.

Follow-up: Why is stack allocation faster than heap allocation?

Q2

■ Easy

What is a memory leak? Give a real-world example.

A memory leak occurs when heap-allocated memory is no longer reachable (no pointers to it) but has not been freed.

Real-world example: A web server that processes each HTTP request by allocating a 4KB buffer for the request body, but forgets to free it after handling the request. After 1 million requests: 4GB leaked, OOM kill.

How to detect: valgrind --leak-check=full, AddressSanitizer with detect_leaks=1, or tools like Heaptrack.

Follow-up: What is the difference between a memory leak and a dangling pointer?

Q3

■ Medium

Explain what happens step-by-step when you call `new int[100]` in C++.

1. Compiler calls **operator new[](400)** (100 ints × 4 bytes).
2. **operator new[]** calls **malloc(400 + overhead)**. The allocator needs extra bytes for metadata (size, flags) stored in a header before the pointer.
3. **malloc** checks its **free-list** for a block ≥ 400 bytes. If none, calls `brk()` or `mmap()` to request pages from the OS (page-aligned, 4KB each).
4. The allocator writes the **allocation size** in the header (needed by `delete[]`).
5. Returns a pointer to the first of the 100 ints. Memory is **NOT zero-initialized** (use `new int[100]()` for that).
6. On failure: throws **std::bad_alloc**.

Follow-up: How does `delete[]` know how many elements to destroy?

Q4

■ Easy

What is the difference between `delete` and `delete[]`?

delete p: for scalars (`new T`). Calls T's destructor once, then frees memory.

delete[] p: for arrays (`new T[N]`). Calls destructors for all N elements (in reverse order), then frees memory.

Wrong match = Undefined Behavior: Using `delete` on `new[]` or `delete[]` on `new` is UB. For scalars with trivial destructors (like `int*`), it may appear to work — but for classes with destructors, it will crash or corrupt memory.

Rule: Every `new` has a `delete`; every `new[]` has a `delete[]`.

Q5

■ Medium

What is a dangling pointer? How is it different from a null pointer?

Dangling pointer: A pointer that points to memory that has already been freed or that has gone out of scope. Reading/writing through it is Undefined Behavior (may read garbage, may crash, may corrupt other data).

Null pointer: A pointer set to `nullptr` (or 0). Dereferencing a null pointer always causes a segmentation fault (predictable, easy to debug).

Code example of dangling pointer:

```
int* p = new int(42);
```

```
delete p;
```

```
cout << *p; // DANGLING — undefined behavior (not null pointer!)
```

Fix: Set to `nullptr` after `delete`: `delete p; p = nullptr;`

Better fix: Use smart pointers (`unique_ptr`) which set themselves to null on destruction.

Follow-up: Can you have a dangling reference in C++? How?

Q6

■ Medium

What are the 4 types of memory errors AddressSanitizer can detect?

1. **Heap-buffer-overflow:** Read/write beyond heap allocation bounds.
2. **Stack-buffer-overflow:** Read/write beyond stack array bounds.
3. **Use-after-free:** Accessing memory after it has been deleted.
4. **Double-free:** Calling delete on the same pointer twice.
5. **Heap-use-after-return:** Using pointer to stack variable after function returns.
6. **Memory leaks:** (with `ASAN_OPTIONS=detect_leaks=1`)

ASan instruments every memory access at compile time. It maintains a **shadow memory** map: for every 8 bytes of application memory, 1 byte of shadow memory tracks validity. ~2x runtime overhead.

Follow-up: Why is ASan preferred over Valgrind for catching stack overflows?

Q7

■ Easy

What is internal vs external fragmentation?

Internal fragmentation: Wasted space INSIDE an allocated block. Caused by alignment requirements. Example: Request 9 bytes, allocator returns 16 bytes (aligned to 8). 7 bytes wasted inside the block.

External fragmentation: Total free memory is sufficient but no single contiguous block is large enough. Example: 100 free blocks of 4 bytes each = 400 bytes total free, but a 100-byte allocation fails.

Solution: Slab/pool allocators eliminate both types by serving fixed-size chunks from pre-allocated slabs.

Follow-up: How does jemalloc or tcmalloc reduce fragmentation compared to glibc malloc?

Q8

■ Easy

What does `ulimit -s` show and why does it matter for recursion?

`ulimit -s` shows the stack size limit in kilobytes (typically 8192 KB = 8 MB on Linux).

Each recursive call consumes stack space for its frame (local variables + saved registers + return address). With 100 bytes per frame and 8 MB stack: ~80,000 recursive calls before SIGSEGV (stack overflow).

For your crawler: Never use recursion for crawling the link graph. The web graph can be billions of nodes deep (circular references). Use an explicit `std::stack` on the heap for DFS instead.

Q9

■ Medium

What is RAII and why does it solve memory management?

RAII = Resource Acquisition Is Initialization. Tie resource lifetime to object lifetime: acquire in constructor, release in destructor. Since destructors are called automatically when scope exits (even on exceptions), resources are always released.

Without RAII (C-style):

```
int* p = new int[N];  
risky(); // throws! delete[] p never called → leak  
delete[] p;
```

With RAII (unique_ptr):

```
auto p = make_unique(N);  
risky(); // throws! destructor still runs → no leak
```

RAII is the core principle behind `unique_ptr`, `shared_ptr`, `lock_guard`, `fstream`, and virtually every well-designed C++ class.

Follow-up: Is RAII possible in C? Why or why not?

Q10

■ Medium

What is the difference between stack overflow and buffer overflow?

Stack overflow: The stack pointer exceeds the stack's size limit (usually 8 MB). Caused by deep recursion or very large stack allocations. OS sends SIGSEGV. Not exploitable in the classic sense — just crashes.

Buffer overflow: A program writes beyond the bounds of a buffer (stack or heap). On the stack, this can overwrite the return address, enabling **arbitrary code execution** (RCE). Classic CVE class.

Stack buffer overflow is a specific type of buffer overflow where the overflowed buffer is on the stack.

Mitigations: Stack canaries (GCC `-fstack-protector`), ASLR, NX bit (non-executable stack), SafeStack (Clang).

Follow-up: What is a stack canary and how does it detect buffer overflows?

Technical / Code Questions

Q11

■ Medium

Trace this code — what does valgrind report? How do you fix it?

```
Code: void f() { char* p = new char[10]; p[10] = 'X'; delete[] p; }
```

Issue 1: `p[10]` is one byte PAST the end of the 10-byte allocation (valid indices: 0–9). This is a **heap-buffer-overflow**.

Issue 2: Valgrind with `--leak-check=full` would report '0 bytes leaked' (`delete[]` is present), but would NOT catch the OOB write reliably. **ASan** will catch it with 'heap-buffer-overflow' immediately.

Fix: Change `p[10]` to `p[9]` (last valid index) or `p[0]-p[9]`.

Follow-up: Why is valgrind less reliable than ASan for OOB detection?

Q12

■ Hard

What is the output of this program? Is there undefined behavior?

Code: `int* p = new int(5); int* q = p; delete p; cout << *q;`

***q after delete p is Use-After-Free — Undefined Behavior.**

In practice it often prints 5 or garbage (the allocator doesn't zero memory on free for performance reasons — only debug allocators do).

The memory pointed to by q has been returned to the free list. The allocator may have written its own bookkeeping data there. A concurrent thread could reallocate it. Output is non-deterministic.

ASan detection: ASAN reports 'heap-use-after-free' at *q read.

Fix: Set `p = nullptr` after delete. q now holds a dangling ptr regardless — never copy raw owning pointers; use `shared_ptr` if you need shared ownership.

Q13

■ Medium

What happens if new fails? How should production code handle it?

By default, `new` throws `std::bad_alloc` if allocation fails.

Handling options:

1. **try/catch (standard):**

`try { int* p = new int[huge]; } catch (bad_alloc& e) { handle(); }`

2. **nothrow version:** `int* p = new(nothrow) int[huge];`

`if (!p) { handle; }` — returns `nullptr` instead of throwing

3. **set_new_handler:** `set_new_handler(my_handler)` — called on failure

In your crawler: If allocating a large HTML buffer fails, log the error and skip that URL — don't crash the entire crawl.

Follow-up: What is the difference between nothrow new and default new?

Q14

■ Hard

Explain the 'Rule of Three' and why it's relevant to memory management.

If a class defines any of: **destructor**, **copy constructor**, **copy assignment operator**, it almost certainly needs to define all three.

Why: Classes that manage heap memory need a destructor to free it. The default copy constructor does a **shallow copy** (copies the pointer, not the data) — two objects now think they own the same memory. When both destructors run, double-free occurs.

Rule of Five (C++11): Also define move constructor and move assignment to avoid unnecessary copies.

Rule of Zero: If possible, use only RAII members (`unique_ptr`, `vector`, `string`) and let the compiler generate all special members correctly.

Follow-up: What problem does the Rule of Zero solve over the Rule of Three?

Q15

■ Medium

What is the difference between `malloc/free` and `new/delete`?

1. **Constructor/Destructor:** `new` calls constructors; `malloc` does NOT. `delete` calls destructor; `free` does NOT. Critical for classes with resources.
 2. **Type safety:** `new` returns typed pointer (`int*`); `malloc` returns `void*` (needs cast).
 3. **Failure:** `new` throws `bad_alloc`; `malloc` returns `nullptr`.
 4. **Operator overloading:** operator `new` can be overloaded per class; `malloc` cannot.
 5. **Mixing is UB:** Never call `free()` on `new` memory or `delete` on `malloc` memory.
- Rule:** Always use `new/delete` in C++. Use `malloc` only when interfacing with C APIs.

Follow-up: Can you call `new` inside `malloc`? What would happen?

Q16

■ Hard

What is a memory-mapped file and when would you use it in your search engine?

`mmap()` maps a file (or anonymous memory) directly into the process's virtual address space. The OS handles paging — no explicit read/write system calls.

Zero-copy I/O: Accessing the mapped region reads from page cache without kernel→userspace copy. For your inverted index files: `mmap` the RocksDB SSTable files → read posting lists without `read()` calls.

When to use in your search engine:

- Reading large inverted index files (GB scale)
- Serving raw HTML from MinIO-style content-addressed storage
- Sharing index data between multiple query worker processes

When NOT to use: Small files (overhead of `mmap` setup > benefit), highly random access patterns (defeats prefetch), write-heavy workloads.

Follow-up: What is the difference between `MAP_PRIVATE` and `MAP_SHARED` in `mmap`?

Q17

■ Medium

You're debugging a crash. The backtrace shows `free(): invalid pointer`. What caused it?

This error from glibc means: `free()` was called with a pointer that is not valid for freeing. Possible causes:

1. **Double-free:** `delete` called twice on same pointer.
2. **Pointer arithmetic:** `free(p+5)` instead of `free(p)` — freeing into middle of block.
3. **Stack address passed to delete:** `int x = 5; delete &x;` — stack, not heap.
4. **Corrupted allocator metadata:** A buffer overflow earlier overwrote the `malloc` header of this block. `Free()` reads the header, finds garbage, aborts.

Debugging: Compile with `-fsanitize=address` and re-run. ASan will pinpoint the exact double-free or OOB write that caused the corruption.

Q18

■ Hard

What is the time complexity of ``new`` and ``delete``? Can you make allocation $O(1)$?

Default glibc malloc:

- Average case: $O(1)$ amortized (free-list lookup, bitmap check)
- Worst case: $O(N)$ where N = number of free blocks (rare, extreme fragmentation)
- delete/free: $O(1)$ for the free itself, $O(N)$ for coalescing adjacent free blocks

Making it $O(1)$ guaranteed:

- **Pool/slab allocator:** Pre-allocate fixed-size objects in a pool array. Alloc = pop from free-list $O(1)$. Free = push to free-list $O(1)$. Zero fragmentation.
- **Arena allocator:** 'Bump pointer' — just increment a pointer. $O(1)$ alloc. Free entire arena at once (no per-object free). Used for request-scoped memory.

For your crawler: Use a pool allocator for URLEntry structs. Each fetch allocates one URLEntry; free after processing. Constant time, no fragmentation.

Follow-up: How does tcmalloc achieve better performance than glibc malloc?

Q19

■ Medium

What is the difference between ``memset``, ``memcpy``, and ``memmove``?

memset(ptr, val, n): Fills n bytes starting at `ptr` with `val`. Use for zero-initialization: `memset(buf, 0, sizeof(buf))`. `val` is cast to unsigned char — only use for byte patterns (0, 0xFF).

memcpy(dst, src, n): Copies n bytes from `src` to `dst`. UNDEFINED BEHAVIOR if regions overlap. Fastest: compiler vectorizes to SIMD.

memmove(dst, src, n): Like `memcpy` but handles overlapping regions correctly (copies through a temporary buffer or direction-aware). Slightly slower.

Rule: Use `memcpy` for non-overlapping; `memmove` for possibly-overlapping. Use `std::copy` or `std::fill` in C++ for type safety (calls constructors).

Follow-up: When does using memcpy on a non-trivially-copyable type cause undefined behavior?

Q20

■ Hard

Explain the double-free exploit. How does it lead to arbitrary code execution?

glibc's ptmalloc stores free-list pointers (fd, bk — forward/backward) inside freed chunks. When you free a chunk twice:

1. First free: chunk added to free-list with valid fd/bk.
2. Second free: glibc's integrity check may pass (older versions); chunk is added again. Free-list now contains a cycle.
3. Next two malloc calls return the same address.
4. Attacker controls write to that address → can overwrite a function pointer (GOT entry) → arbitrary code execution.

Modern mitigations:

- glibc 2.26+: tcache double-free detection (checks prev pointer)
- ASan: immediate abort on double-free with full stack trace
- SAFE_HEAP in Emscripten, PartitionAlloc in Chrome

For your crawler: Double-free = crash at best, RCE at worst if attacker can control malloc patterns. Use `unique_ptr` exclusively.

Scenario & System Design Questions

Q21

■ Medium

Your crawler runs for 2 hours then gets OOM-killed. How do you diagnose and fix it?

Step 1 — Reproduce: Run with `valgrind --tool=massif` to get heap profile. Or add `MALLOC_TRACE` to dump alloc/free calls.

Step 2 — Identify the leak source: `massif-visualizer` shows heap growth over time. Common sources: URL string buffer not freed after crawl, HTML body buffer kept in memory, accumulating parsed links not cleared.

Step 3 — Systematic fix:

- Every crawled page: allocate buffer → fetch → parse → index → FREE buffer.
- Use arena allocator: allocate per-crawl-cycle arena, reset at end of cycle.
- Use RAI: wrap all buffers in `unique_ptr` → automatic free on scope exit.

Step 4 — Verify: Re-run with `valgrind --leak-check=full`. Monitor `/proc/PID/status` (VmRSS field) to confirm stable memory usage.

Follow-up: What is a memory watermark and how would you track it over time?

Q22

■ Hard

Design a custom allocator for your URL queue. What properties do you want?

Requirements for URL queue allocator:

- Fixed-size objects (URLEntry struct: ~200 bytes) → pool allocator ideal.
- High allocation/deallocation rate (millions/day) → $O(1)$ both operations.
- Thread-safe (64 worker threads) → per-thread pools with central fallback.
- Memory-efficient: no fragmentation, predictable footprint.

Design — Two-tier pool:

- Per-thread cache: each thread has a fixed free-list of 256 URLEntry slots. Alloc/free never acquires a lock (thread-local).
- Global pool: when thread cache is empty, refill from global pool (lock). When thread cache is full, return batch to global pool.
- This is exactly how `tcmalloc` and `jemalloc` work — per-thread caches.

Result: 0 lock contention for >95% of allocations. No fragmentation. Predictable memory usage.

Follow-up: How does this relate to how jemalloc implements thread-local caches?

Q23

■ Medium

When should you prefer stack allocation over heap allocation?**Prefer stack when:**

- Size is known at compile time and small (< a few KB)
- Object lifetime = current function scope
- Performance-critical path (avoid allocator overhead)
- No sharing between threads needed

Examples in your crawler:

- HTTP status code (int): stack
- Local URL parse result (struct ~128 bytes): stack
- DNS cache entry (stack during lookup, heap in cache): mixed

Prefer heap when:

- Size determined at runtime (e.g., HTML body size from Content-Length header)
- Lifetime exceeds function scope (URL passed to thread pool)
- Large data (> few KB — risks stack overflow)
- Shared between threads or returned to caller

Follow-up: Can you use the stack for all allocations in a crawler? Why or why not?

Q24

■ Hard

How does the Linux kernel manage memory for your process? Explain virtual memory.

Linux uses **virtual memory**: each process sees a private 48-bit address space (on x86-64). Physical RAM is shared via the **page table**.

Key regions in your process's virtual address space:

- 0x0 – text (code), data (globals), BSS (zero-init globals)
- Heap: starts after BSS, grows upward via `brk()`
- `mmap` region: grows downward — for large malloc, shared libs, file mappings
- Stack: starts at top of user space, grows downward

Page fault: When you access a new heap page, the page isn't in RAM yet. CPU triggers a page fault → kernel handles it → maps a physical frame → returns to userspace. This is why first access is slower than subsequent accesses.

OOM killer: When RAM is exhausted, kernel selects and kills a process (score based on memory usage + priority). Your 2-hour crawler OOM = this.

Follow-up: What is the difference between `VmRSS` and `VmSize` in `/proc/PID/status`?

Q25

■ Expert

What is memory alignment and why does it matter for performance?

A value is **aligned** if its address is a multiple of its size. Example: a 4-byte int should be at an address divisible by 4 (0x1000, 0x1004...).

Why CPUs require alignment: x86 handles misaligned access but with a penalty (1-2 extra cycles). ARM: misaligned access triggers a bus error (hardware exception) on older versions. SIMD instructions (SSE, AVX) require 16 or 32-byte alignment.

Struct padding:

```
struct Bad { char c; int i; }; // sizeof = 8 (3 bytes padding before i)
```

```
struct Good { int i; char c; }; // sizeof = 8 (3 bytes padding after c)
```

```
struct Best { int i; int j; char c; }; // sizeof = 12 (optimized layout)
```

For your index: Align posting list arrays to 32 bytes for AVX-accelerated BM25 scoring (SIMD processes 8 floats simultaneously).

Follow-up: How does `alignas()` work in C++11 and when would you use it?

Q26

■ Hard

A colleague says 'just use smart pointers everywhere and never worry about memory again.' Is this correct?**Mostly right, but with important caveats:**

1. **Cycles with `shared_ptr`:** Two objects holding `shared_ptr` to each other create a reference cycle → neither ever freed. Fix: use `weak_ptr` to break cycles.
2. **Performance:** `shared_ptr` has ~3x overhead vs raw pointer (atomic ref count). `unique_ptr` has zero overhead. Choose wisely.
3. **Array types:** `unique_ptr` vs `array` — easy to mix up.
4. **Interop with C APIs:** You still need `.get()` to pass to C functions, and must not delete the raw pointer received from the smart pointer.
5. **Stack allocations:** Smart pointers are for heap. Stack variables need no smart pointers — they're auto-managed by scope.

Correct rule: Use `unique_ptr` by default, `shared_ptr` when shared ownership is semantically correct, raw pointers for non-owning references.

Advanced / Expert Questions

Q27

■ Expert

Explain the glibc malloc internals: free-list, bins, and tcache.

glibc ptmalloc2 architecture:

- **Chunk:** Basic allocation unit. Header stores: size, prev_size, flags (IS_MMAPPED, PREV_INUSE).
- **Bins:** Free chunks organized by size:
 - Fast bins (16–80 bytes): LIFO, no coalescing, for small fast allocs
 - Small bins (< 512 bytes): doubly-linked, sorted by size
 - Large bins (>= 512 bytes): doubly-linked, sorted + skip list
 - Unsorted bin: newly freed chunks land here first
- **tcache (glibc 2.26+):** Per-thread cache of 7 chunks per size class. Alloc/free never touches a lock. Fastest path.
- **Arena:** Multiple arenas for multi-threaded apps. Each thread gets its own arena (up to `cpu_count * 8` arenas). Reduces lock contention.

For your crawler: 64 threads → 64 thread-local tcaches → near-zero contention on malloc/free. This is why modern malloc is thread-scalable.

Follow-up: What is the wilderness chunk (top chunk) in ptmalloc?

Q28

■ Expert

What is AddressSanitizer's shadow memory technique?

ASan uses a **shadow memory map**: for every 8 bytes of application memory, 1 byte of shadow memory encodes validity:

- Shadow byte = 0: all 8 bytes accessible (valid)
- Shadow byte = k (1-7): first k bytes accessible, rest are redzone
- Shadow byte = negative (0x80-0xFF): entire 8-byte region is poisoned (redzone, freed mem)

Instrumentation: Every memory access (load/store) is preceded by a shadow check:

```
shadow_addr = (app_addr >> 3) + 0x7fff8000;
```

```
shadow_byte = *shadow_addr;
```

```
if (shadow_byte != 0 && ...) ASan_report();
```

Redzones: ASan inserts 32-byte poisoned redzones before/after each allocation. OOB access into the redzone → shadow check fails → report.

Freed memory: On delete, ASan poisons the shadow bytes and puts the chunk in a quarantine (not immediately returned to allocator) → catches use-after-free.

Follow-up: Why does ASan need to reserve 20 TB of virtual address space on 64-bit Linux?

Q29

■ Expert

How would you implement a garbage collector for C++? What approach would you take?**Conservative GC (Boehm-Demers-Weiser):**

- Replaces malloc with gc_malloc. Periodically scans all memory (stack, globals, heap) for values that look like valid heap pointers.
- 'Conservative' because it can't distinguish a genuine pointer from an integer that happens to equal a heap address — may retain unreachable objects ('false positives').
- Used by: Mono runtime, older versions of Mozilla.

Precise GC (not standard C++): Would require cooperation from the compiler to emit pointer maps (knowing exactly which stack/heap locations are pointers).

Reference Counting (std::shared_ptr): Not a GC but achieves similar goals. Drawback: cannot collect cycles → need weak_ptr.

Why C++ avoids GC: Deterministic destruction (RAII), no stop-the-world pauses, predictable latency — critical for real-time systems and your search engine's <100ms SLA.

Follow-up: What is a tri-color mark-and-sweep GC? How does it achieve concurrent collection?

Q30

■ Expert

What is the PROT_NONE guard page trick and how does Linux implement stack growth?

Guard page: The OS places a PROT_NONE (no read/write/execute) page just below the stack's current bottom. When stack overflows into this page, the CPU generates a page fault → OS sees it's a stack extension request OR a genuine overflow (stack already at max size).

Stack growth mechanism (Linux):

1. Stack starts at a high virtual address, one page allocated.
2. Guard page placed one page below the stack bottom.
3. Stack grows downward (push, function calls). When accesses the guard page:
4. Page fault handler: checks if stack can grow (VMA limit, ulimit -s). If yes: extend VMA, allocate new page, return to user. If no: send SIGSEGV (stack overflow).

Why this matters:

- Default stack is only 1 page initially — it grows lazily. alloca(8MB) on first use is as expensive as allocating 2048 pages (page faults).
- In high-performance C++: pre-touch stack pages at thread start to avoid latency spikes from page faults during crawl.

Follow-up: What is a huge page and when would you use it for your inverted index storage?

■ Day 2 Summary — What You Must Know Cold

STACK: Fast (~1ns alloc), LIFO, automatic, 8MB limit, thread-private. Destroyed on scope exit. Overflow via deep recursion = SIGSEGV.

HEAP: Flexible (runtime size), manual, shared, slow (~100ns). Leak = memory never returned. Always pair `new[]` with `delete[]`.

VALGRIND: `--leak-check=full` for post-run leak audit. 10–50x overhead.

ASAN: `-fsanitize=address` for heap/stack OOB, UAF, double-free. 2x overhead.

FRAGMENTATION: Internal (padding) vs External (no contiguous block). Fix: pool allocators.

RULE: In production C++, use `unique_ptr` and let RAI handle everything. Raw owning pointers are a code smell after Day 3.

NEXT STEPS

Day 3 builds directly on today: `RAII` + `unique_ptr` + `shared_ptr` eliminate all the manual memory management you practiced here. Every pattern you built today (dynamic array, linked list, manual free) will be rewritten with smart pointers. The muscle memory of knowing WHEN memory leaks (and WHY smart pointers prevent it) comes from today's exercises.

Resources to Explore Further:

- ◆ learncpp.com — Dynamic Memory Allocation with `new/delete`
- ◆ MIT OCW 6.172 — Memory lecture (Performance Engineering)
- ◆ Google AddressSanitizer wiki — github.com/google/sanitizers
- ◆ Valgrind Quickstart — valgrind.org/docs/manual/quick-start.html
- ◆ What Every Programmer Should Know About Memory — Ulrich Drepper (PDF)
- ◆ The Chernobyl — Stack vs Heap Memory (YouTube)